



2026 3-1 응개

▼ 기획서

1. 프로젝트 개요

- **프로젝트명(가칭):** DevClass (개발자 인강 플랫폼)
- **목표:** 학생(수강생)이 강의를 검색/신청/수강하고, 강사(판매자)가 강의를 등록/관리하며, 운영 단계에서 구독·알림·관측성까지 갖춘 **단계별 확장형 웹 서비스**를 구현한다.
- **단계별 제공 가치**
 - **P1 (학생용 / 인증 제외):** CRUD + DB 기반으로 "강의 탐색 → 상세 보기 → 수강 등록 → 진도 기록" 핵심 루프 완성
 - **P2 (강사용 / 인증+권한):** JWT + RBAC로 "강의 생성/수정/삭제, 수강자 조회, 관리자 기능" 제공
 - **P3 (운영형 / 외부연동+관측성):** 구독(이메일/디스코드)·스케줄러·Webhook·모니터링·보안·성능·배포·결제모듈까지 운영 수준으로 강화

2. 문제정의

- 학습자는 강의를 "찾기 쉽고", "수강 상태를 기록"할 수 있어야 한다.
- 강사는 강의/콘텐츠를 "쉽게 등록·수정"하고 "판매/운영 지표(수강자 수 등)를 확인"해야 한다.
- 운영자는 장애/보안/성능 문제를 "발견·추적·대응"할 수 있어야 한다.

3. 대상 사용자(페르소나)

- **학생(수강생):** 원하는 강의를 빠르게 찾고 수강 등록 후 학습 진도를 기록하고 싶다.
- **강사(판매자):** 강의 정보를 등록하고 강의를 운영하고 싶다.
- **관리자:** 사용자 신고 및 강의 신고를 관리하고, 문제가 있는 강의를 삭제하거나 조치한다.

4. 핵심 기능 범위(MVP → 확장)

- **P1 MVP(인증 제외)**
 - 강의 목록/검색/정렬, 강의 상세
 - 수강 등록(Enrollment) / 수강 취소
 - 진도(Progress) 기록(강의별 % 또는 "마지막 본 레슨")
- **P2 확장(인증/권한)**
 - 회원가입/로그인(JWT)
 - 역할: STUDENT / INSTRUCTOR / ADMIN
 - 강사 전용: 강의 CRUD, 커리큘럼(레슨) CRUD, 수강자 조회
 - 관리자 전용
 - 신고된 강의 관리
 - 강의 삭제
 - 사용자 관리
- **P3 운영형**
 - 구독/알림: 새 강의 등록 알림(할 수 있다면 디스코드 아니면 이메일 중 1개)
 - 스케줄러: 주간 인기 강의 집계, 만료 토큰/임시데이터 정리
 - Webhook: 결제/구독 이벤트 수신(또는 디스코드 webhook 발송)
 - Observability: 구조화 로그, 헬스체크, 슬로우쿼리/메트릭, Sentry
 - 보안: SQLi/XSS 방어 체크리스트 반영
 - 성능: 인덱스/페이징, 프론트 렌더 최적화
 - 배포: Docker + EC2 + GitHub Actions CI/CD
 - 결제모듈(범위 조정 가능): "결제 연동(샌드박스) + 결제내역 저장" 수준

5. 비기능 요구(중요)

- **DBMS(MySQL) 필수**, 마이그레이션(Flyway 권장)
- **API 문서(OpenAPI) 지속 업데이트**
- **테스트 최소 세트**: 백엔드 통합 1~2, 프론트 테스트/또는 e2e 1개
- **보안 기본**: 입력 검증, 권한 체크, 에러 메시지 최소화, CORS/CSRF 고려(Next + JWT 구조에서 정리)

6. 성공 기준(완료 정의)

- P1: 인증 없이도 강의 → 수강 → 진도 기록 플로우가 로컬에서 완전 동작
- P2: 로그인 후 역할별로 API 접근이 제한되고 UI도 분기됨
- P3: 알림/스케줄러/관측성/배포까지 "운영 가능한 형태"로 데모 가능

7. 벤치마킹 사이트

- 인프런 ★
- 노마드코더
- 코드잇

▼ 개발계획서

1. 기술 스택

- **Frontend:** Next.js, Tailwind CSS
- **Backend:** Spring Boot (REST API)
- **DB:** MySQL
- **Infra/DevOps:** Docker, GitHub, GitHub Actions, EC2
- **Observability:** 구조화 로그(JSON), 헬스체크/메트릭, Sentry(프론트/백 중 1개 이상)

2. 개발 환경/규칙(운영 방식)

- **브랜치 전략:** GitHub Flow
 - `main` (배포 가능), `feature/*` (기능), PR 리뷰 후 merge
- **커밋 규칙:** Conventional Commits
 - 예: `feat: add course list api`, `fix: handle enrollment duplicate`
- **PR/이슈 템플릿:** 목적/변경점/테스트/스크린샷/체크리스트
- **Definition of Done(DoD)**
 - 기능: 요구사항(AC) 충족
 - 문서: OpenAPI 반영 + 간단 사용법 기록
 - 품질: 최소 테스트 통과 + lint/build 통과

3. 아키텍처(요약)

- Next.js(웹) ↔ Spring Boot(API) ↔ MySQL
- P3에서: Scheduler(서버 내부 cron) + 외부 알림(Discord/Email) + Sentry/Logs

4. 데이터/도메인(초안)

- User(사용자) / Course(강의) / Lesson(레슨) / Enrollment(수강) / Progress(진도) / Review(후기) / Payment(결제) / Subscription(구독)

5. 일정/마일스톤(타임스탬프 기반)

| 작성일: 2026-03-09, 학교 주차 계획을 주차 단위 목표 + 산출물로 고정

[1~4주차] P1 (학생용, 인증 제외) — 4/3 발표(로컬)

- 1주차: 설계/기획 & 백/프 개발 시작
 - 산출물: 기획서, 개발계획서, 요구사항명세서, API Draft, ERD 초안
 - 백엔드: Course/Enrollment/Progress 기본 CRUD
 - 프론트: 목록/상세/수강/진도 UI 라우팅
- 2주차: 기능 확장 + 데이터 일관성
 - 검색/페이징, 예외처리 표준, 마이그레이션(Flyway)
- 3주차: P1 완료/정리
 - 테스트 최소, OpenAPI 최신화, 발표 시나리오 정리
- 4주차: 4/3 발표 준비(로컬)
 - 최종 점검(로컬 실행/DB/주요 플로우) + 발표 리허설

[5~10주차] P2 (인증+권한, 강사용) — 5/8 발표(로컬)

- 5주차: 설계/기획 + 인증 구현 착수
 - P2 요구사항 정리(역할: 학생/강사/관리자)
 - JWT 인증 방식 + 권한 정책(RBAC) 정리
 - API Draft + 화면 흐름(로그인/강사페이지) 정리
 - (가능하면) 백엔드에 auth 프로젝트 뼈대/엔드포인트 틀 잡기

- **6주차: 인증 기능 구현(핵심 완료 목표)**
 - 백엔드: 회원가입/로그인, JWT 발급/검증 구현
 - 프론트: 로그인/회원가입 화면 + 로그인 상태 처리
 - 기본 권한 적용(로그인 안 하면 접근 제한)
- **7주차: P1 + P2 통합**
 - 프론트 보호 라우트 적용(역할별 UI 분기)
 - 학생 기능(P1)과 강사 기능(P2) 한 프로젝트로 연결
 - 통합 동작 점검(핵심 플로우 위주)
- **8주차: 강사용 기능 구현**
 - 강사 기능: 강의/레슨 등록·수정·삭제(CRUD)
 - 역할별 접근 제어 강화(강사만 가능, 관리자는 전체 가능 등)
- **9주차: 발표용 마감/정리**
 - 버그 수정, 예외 케이스 점검
 - 문서(OpenAPI/요구사항) 최신화
 - 발표 데모 시나리오 정리 + 리허설
- **10주차: 최종 테스트 + 발표 연습 + 안정화**
 - 전체 기능 최종 테스트(인증/권한/강사CRUD/학생기능)
 - 발표 리허설 + 시연 흐름 고정
 - (시간 남으면) 간단 개선/정리

[11~16주차] P3 (운영형) — 6/29 최종(배포)

- **11주차: 운영 설계/정리 + 외부연동 착수**
 - 운영 관점 요구사항 정리(보안/테스트/관측성 포함)
 - 배포 방식/CI-CD 방향 정리
 - 외부연동(알림/결제/웹훅) 범위 확정
 - (가능하면) 알림/구독 또는 webhook 중 1개 “틀”이라도 착수
- **12주차: 성능/품질 개선**
 - 느린 API/쿼리 개선(페이징, 인덱스 등)

- 프론트 렌더링/데이터 로딩 개선
- 코드 정리(리팩토링) + CI/CD 보완
- **13주차: 외부연동 + 배치/스케줄러 구현**
 - 알림/구독 기능 1개 구현(이메일 or 디스코드)
 - 스케줄러 작업 1개 구현(집계/정리 등)
 - webhook 1개 구현(수신 또는 발신)
- **14~15주차: 배포 준비 + 운영 리허설 + 결제(최소)**
 - EC2 배포 환경 세팅 + Docker 정리
 - CI/CD로 배포 자동화 연결
 - 장애/실패 상황 리허설(알림 실패, DB 문제 등)
 - 결제모듈은 "최소 범위"로 적용(가능한 수준으로)
- **16주차: 최종 테스트 + 발표 연습 + 데모 고정**
 - 최종 회귀 테스트(배포 환경에서 학생/강사 플로우)
 - 포트폴리오 문서 정리
 - 최종 리허설 후 배포 상태로 시연
 - 최종발표(6/29)

2.6 리스크/대응

- **일정 과부하:** 결제는 "샌드박스 + 결제내역 저장"으로 최소화
- **권한 설계 복잡:** P2에서 Role 3개(STUDENT/INSTRUCTOR/ADMIN)로 제한
- **외부연동 실패:** Discord webhook처럼 구현 난이도 낮은 채널 우선
- **성능 튜닝:** 인덱스/페이징/캐시(가능하면)부터

▼ 요구사항명세서

1. 용어 정의

- **Course:** 강의(상품/학습 단위)
- **Lesson:** 강의 내 레슨(회차/챕터)
- **Enrollment:** 수강 등록(학생이 강의를 듣는 상태)
- **Progress:** 수강 진도(레슨 기반 또는 퍼센트)

- **RBAC:** 역할 기반 접근 제어
-

2. P1 요구사항 (학생용 / 인증 제외)

기능 요구사항 FR-P1

1. 강의 목록 조회

- 설명: 전체 강의 목록을 페이지 단위로 조회
- AC:
 - 페이지/사이즈 파라미터로 페이징 가능
 - 정렬(최신순/인기순(선택)) 가능

2. 강의 검색

- 설명: 키워드로 강의 제목/설명 검색
- AC:
 - 검색어 없으면 전체 목록
 - 검색어 있으면 필터링 결과

3. 강의 상세 조회

- 설명: 강의 상세(설명, 강사명(문자열), 레슨 목록(선택)) 표시
- AC:
 - 존재하지 않는 강의면 404

4. 수강 등록/취소

- 설명: 특정 사용자(인증 대신 임시 userId로 처리)가 강의를 수강 등록
- AC:
 - 중복 수강 등록 방지(409 등)
 - 취소 시 상태 변경 또는 삭제 정책 중 택1(문서로 고정)

5. 진도 기록

- 설명: 수강 중인 강의의 진도 저장/조회
- AC:
 - 수강하지 않은 강의에 진도 기록 불가
 - 진도 조회 시 현재 % 또는 마지막 레슨 정보 반환

비기능 요구사항 NFR-P1

- MySQL 사용, 마이그레이션 적용(Flyway)
 - 입력값 검증(빈 문자열/길이 제한/범위)
 - 예외 응답 형식 표준화(에러코드/메시지/타임스탬프)
-

3. P2 요구사항 (JWT + 권한)

기능 요구사항 FR-P2

1. 회원가입/로그인

- AC:
 - 비밀번호는 해시 저장(BCrypt)
 - 로그인 성공 시 Access Token 발급(JWT)

2. 권한 모델

- Roles: **STUDENT** , **INSTRUCTOR** , **ADMIN**
- AC:
 - 강의 생성/수정/삭제는 INSTRUCTOR 이상만 가능
 - ADMIN은 전체 관리 가능

3. 강사 대시보드(기능)

- AC:
 - 본인 강의 목록 조회
 - 본인 강의 수강자 수 조회(간단 집계)

4. 강의 신고

- 설명
 - 사용자는 부적절한 강의를 신고할 수 있다.
- AC
 - 동일 사용자는 동일 강의를 중복 신고할 수 없다
 - 신고 사유 입력 가능

5. 신고 내역 조회 (관리자)

- 설명

- 관리자는 신고된 강의 목록을 확인할 수 있다.
- AC
 - 강의별 신고 횟수 표시
 - 신고 상세 조회 가능
- 6. 강의 삭제 (관리자)
 - 설명
 - 관리자는 신고된 강의를 삭제할 수 있다.
 - AC
 - 강의 삭제 시 수강생에게 알림 발송

보안 요구사항 SEC-P2

- 인증 없는 요청 차단(보호 API)
 - 토큰 만료 처리(만료 시 401)
 - 권한 부족 시 403
-

4. P3 요구사항 (운영형/외부연동/관측성/성능/배포)

기능 요구사항 FR-P3

1. 구독/알림

- AC:
 - “새 강의 등록” 이벤트 발생 시 구독자에게 알림 발송
 - 실패 시 재시도(최소 1회) 또는 실패 로그 기록

2. 스케줄러

- 예: 매일/매주 인기 강의 집계, 오래된 작업 로그 정리
- AC:
 - 작업 실행 이력 저장(job_runs 테이블 등)

3. Webhook

- AC:
 - 외부에서 이벤트 수신(결제 등) 또는 외부로 이벤트 발송(디스코드 webhook)

4. 관측성

- AC:
 - 헬스체크 엔드포인트 제공
 - 구조화 로그(요청 ID, 사용자 ID, 응답시간) 남김
 - Sentry로 프론트/백 에러 수집 중 1개 이상

5. 보안

- AC:
 - SQL Injection 방어: Prepared Statement/JPA 사용, 동적쿼리 안전
 - XSS 방어: 프론트 출력 인코딩, dangerouslySetInnerHTML 지양

6. 성능

- AC:
 - 목록 API 페이징 + 인덱스 적용
 - 프론트: 불필요한 렌더 줄이기(메모이제이션/서버컴포넌트 전략 등 중 택)

7. 배포

- AC:
 - Docker 기반으로 EC2 배포
 - GitHub Actions로 CI/CD 파이프라인 구성

▼ 기능 명세서

▼ 강의 탐색 기능

▼ 강의 목록 조회

사용자는 **전체 강의 목록을 확인**할 수 있다.

강의는 **최신순 또는 인기순으로 정렬**할 수 있다.

페이지 단위로 강의 목록을 조회할 수 있다.

입력

- page
- size
- sort

출력

- 강의 목록
- 총 강의 수
- 현재 페이지 정보

▼ 강의 검색

사용자는 **키워드를 입력하여 강의를 검색**할 수 있다.

검색 대상

- 강의 제목
- 강의 설명
- 검색 결과가 없다면 빈 리스트 반환

▼ 강의 상세 조회

사용자는 강의의 **상세 정보를 확인**할 수 있다.

포함 정보

- 강의 제목
- 강의 설명
- 강사 이름
- 가격
- 평점
- 레슨 목록
- 수강자 수
- 생성일
- 존재하지 않는 강의 요청 시 404 반환

▼ 수강 기능

▼ 수강 등록

사용자는 **강의를 수강 등록**할 수 있다.

제약 조건

- 이미 수강한 강의는 중복 등록 불가
- 강의가 존재해야 한다

등록 성공 시 Enrollment 생성

▼ 수강 취소

사용자는 **수강 중인 강의를 취소**할 수 있다.

정책

- soft delete 또는 상태 변경 방식 사용

▼ 내 수강 목록 조회

사용자는 **자신이 수강 중인 강의를 조회**할 수 있다.

출력 정보

- 강의 제목
- 진도율
- 수강 시작일

▼ 신고 기능

▼ 강의 신고

사용자는 **부적절한 강의를 신고**할 수 있다.

신고 사유 예시

- 부적절한 콘텐츠
- 저작권 침해
- 잘못된 강의 내용
- 스팸 강의
- 기타

제약 조건

- 동일 사용자는 동일 강의를 중복 신고할 수 없다

입력

- courseld
- reason
- description

출력

- 신고 등록 완료 메시지

▼ 신고 내역 조회 (관리자)

관리자는 **전체 신고 내역을 확인할 수 있다.**

관리자 페이지에서는 **강의 기준으로 신고가 집계되어 표시된다**

예시

```
React 입문 강의
신고 횟수 : 5

펼치기
└ user1 : 저작권 침해
└ user3 : 스팸
└ user7 : 잘못된 강의
```

출력 정보

- 강의 제목
- 신고 횟수
- 신고 사용자 목록
- 신고 사유

▼ 학습 진도 기능

▼ 진도 조회

사용자는 **특정 강의의 학습 진도를 조회**할 수 있다.

출력

- 진도율
- 마지막 시청 레슨
- 마지막 업데이트 시간

▼ 진도 업데이트

사용자는 **학습 진도를 저장**할 수 있다.

조건

- 수강하지 않은 강의는 진도 저장 불가

▼ 강사 기능

▼ 강의 생성

강사는 **새로운 강의를 등록**할 수 있다.

입력 정보

- 제목
- 설명
- 가격

▼ 강의 수정

강사는 **자신이 만든 강의를 수정**할 수 있다.

수정 가능 항목

- 제목
- 설명
- 가격

▼ 강의 삭제

강사는 **자신의 강의를 삭제**할 수 있다.

관리자는 모든 강의를 삭제 가능

▼ 레슨 관리

강사는 **강의에 레슨을 추가**할 수 있다.

레슨 정보

- 레슨 제목
- 영상 URL
- 레슨 순서

▼ 수강자 조회

강사는 **자신의 강의 수강자 목록을 조회**할 수 있다.

출력 정보

- 사용자 이름
- 수강 시작일
- 진도율

▼ 사용자 인증 기능

▼ 회원가입

사용자는 **계정을 생성**할 수 있다.

입력

- 이메일
- 비밀번호
- 이름

(비밀번호는 BCtype로 암호화 저장)

▼ 로그인

사용자는 로그인할 수 있다.

로그인 성공 시 JWT 토큰 발급

▼ 관리자 기능

▼ 사용자 관리

관리자는 전체 사용자 목록을 조회할 수 있다.

관리자는 사용자를 비활성화할 수 있다.

▼ 강의 관리

관리자는 강의를 승인하거나 숨김 처리할 수 있다.

▼ 알림 기능

▼ 강의 알림 구독

사용자는 특정 강사의 새 강의 알림을 구독할 수 있다.

▼ 새 강의 알림 발송

새 강의 등록 시 구독자에게 알림이 발송된다.

알림 채널

- 이메일
- 디스코드 webhook

▼ API 명세서

1. 공통 규칙

- Base URL(로컬): `/api`
- Content-Type: `application/json`

인증 방식 (P2 이후)

Authorization: Bearer <JWT>

공통 에러 응답 형식

```
{
  "code": "ERROR_CODE",
  "message": "에러 메시지",
  "timestamp": "2026-03-10T12:00:00",
  "path": "/api/..."
}
```

P1 API (인증 제외)

2. Courses

2.1 강의 목록 조회

GET `/api/courses?page=1&size=20&query=&sort=latest`

Response

```
{
  "items": [
    {
      "courseId": 1,
      "title": "React 입문",
      "instructorName": "김개발",
      "price": 30000,
      "ratingAvg": 4.8,
      "createdAt": "2026-03-10"
    }
  ],
  "page": {
    "page": 1,
    "size": 20,
    "total": 120
  }
}
```

2.2 강의 상세 조회

GET `/api/courses/{courseId}`

Response

```
{
  "courseId":1,
  "title":"React 입문",
  "description":"React 기본 강의",
  "instructorName":"김개발",
  "lessons": [
    {
      "lessonId":1,
      "title":"React 소개",
      "order":1
    }
  ],
  "createdAt":"2026-03-10"
}
```

에러

404 NOT_FOUND

2.3 강의 생성 (P2에서 보호)

POST `/api/courses`

Body

```
{
  "title":"Spring Boot 입문",
  "description":"Spring Boot 기초 강의",
  "price":50000
}
```

Response

```
{
  "courseId":10
}
```

2.4 강의 수정 (P2에서 보호)

PUT `/api/courses/{courseId}`

Body

```
{
  "title": "Spring Boot 완전 정복",
  "description": "내용 수정",
  "price": 60000
}
```

2.5 강의 삭제

DELETE `/api/courses/{courseId}`

권한

INSTRUCTOR (본인 강의)
ADMIN

Response

204 No Content

3. Enrollments

(P1에서는 userId 임시 전달)

3.1 수강 등록

POST `/api/enrollments`

Body

```
{
  "userId": 1,
  "courseId": 10
}
```

Response

```
{
  "enrollmentId":5,
  "userId":1,
  "courseId":10,
  "status":"ENROLLED",
  "enrolledAt":"2026-03-10"
}
```

에러

409 ALREADY_ENROLLED

3.2 내 수강 목록

GET `/api/users/{userId}/enrollments?page=1&size=20`

Response

```
{
  "items": [
    {
      "courseId":10,
      "title":"Spring Boot 입문",
      "progressPercent":40,
      "enrolledAt":"2026-03-10"
    }
  ]
}
```

3.3 수강 취소

DELETE `/api/enrollments/{enrollmentId}?userId=1`

Response

204 No Content

에러

403 FORBIDDEN

4. Progress

4.1 진도 조회

GET `/api/users/{userId}/courses/{courseId}/progress`

Response

```
{
  "courseId": 10,
  "userId": 1,
  "percent": 60,
  "lastLessonId": 5,
  "updatedAt": "2026-03-10"
}
```

4.2 진도 업데이트

PUT `/api/users/{userId}/courses/{courseId}/progress`

Body

```
{
  "percent": 60,
  "lastLessonId": 5
}
```

에러

```
400 INVALID_PROGRESS
403 NOT_ENROLLED
```

P2 확장 (인증 / 권한)

5. Auth

5.1 회원가입

POST `/api/auth/register`

```
{
  "email": "user@test.com",
  "password": "1234",
  "name": "홍길동"
}
```

5.2 로그인

POST `/api/auth/login`

Response

```
{
  "accessToken": "jwt-token",
  "userId": 1,
  "role": "STUDENT"
}
```

6. Instructor API

6.1 내 강의 목록

GET `/api/instructor/courses`

```
[
  {
    "courseId": 10,
    "title": "Spring Boot 입문",
    "studentCount": 120
  }
]
```

6.2 수강자 조회

GET `/api/instructor/courses/{courseId}/students`

```
[
  {
    "userId": 3,
    "name": "김학생",
    "progressPercent": 50
  }
]
```

```
}  
]
```

7. Report API (강의 신고)

7.1 강의 신고

POST `/api/reports`

```
{  
  "courseId":10,  
  "reason":"COPYRIGHT",  
  "description":"저작권 문제 의심"  
}
```

Response

```
{  
  "reportId":1,  
  "courseId":10,  
  "userId":3,  
  "createdAt":"2026-03-10"  
}
```

8. Admin API

8.1 신고 목록 조회

GET `/api/admin/reports`

```
[  
  {  
    "courseId":10,  
    "courseTitle":"Spring Boot 입문",  
    "reportCount":4  
  }  
]
```

8.2 신고 상세 조회

GET `/api/admin/courses/{courseId}/reports`

```
[
  {
    "userId": 3,
    "reason": "COPYRIGHT",
    "description": "저작권 의심",
    "createdAt": "2026-03-10"
  }
]
```

8.3 강의 삭제

DELETE `/api/admin/courses/{courseId}`

```
{
  "deleteReason": "저작권 문제"
}
```

동작

1. 강의 삭제
2. 수강생 조회
3. 수강생에게 삭제 알림 발송

Response

204 No Content

9. Notification API

9.1 내 알림 조회

GET `/api/users/{userId}/notifications`

```
[
  {
    "notificationId": 1,
    "title": "강의 삭제 안내",
    "message": "수강 중이던 강의가 삭제되었습니다.",
  }
]
```

```
"createdAt": "2026-03-10"
  }
]
```

10. 운영 API (P3)

10.1 알림 구독

POST `/api/subscriptions`

10.2 결제 Webhook

POST `/api/webhooks/payment`

10.3 헬스 체크

GET `/api/health`

```
{
  "status": "UP"
}
```

10.4 작업 이력 조회

GET `/api/admin/jobs/runs`

▼ 사용자 시나리오

1. 학생 시나리오 (P1)

1. 사용자가 강의 목록을 조회한다
2. 검색을 통해 강의를 찾는다
3. 강의 상세 페이지를 확인한다
4. 강의를 수강 등록한다
5. 강의를 시청한다
6. 진도 30%를 저장한다
7. 다음날 접속 후 진도 60%로 갱신한다

2. 강사 시나리오 (P2)

1. 강사가 회원가입을 한다
 2. 로그인한다
 3. 강의를 생성한다
 4. 강의 레슨을 추가한다
 5. 강의를 공개한다
 6. 수강자 수를 확인한다
-

3. 강의 신고 시나리오

1. 학생이 강의를 시청한다
 2. 부적절한 강의라고 판단한다
 3. 신고 버튼을 클릭한다
 4. 신고 사유를 입력한다
 5. 신고가 등록된다
-

4. 관리자 신고 처리 시나리오

1. 관리자가 관리자 페이지에 접속한다
 2. 신고된 강의 목록을 확인한다
 3. 특정 강의를 선택한다
 4. 신고 상세 내용을 확인한다
 5. 강의를 삭제하거나 유지한다
-

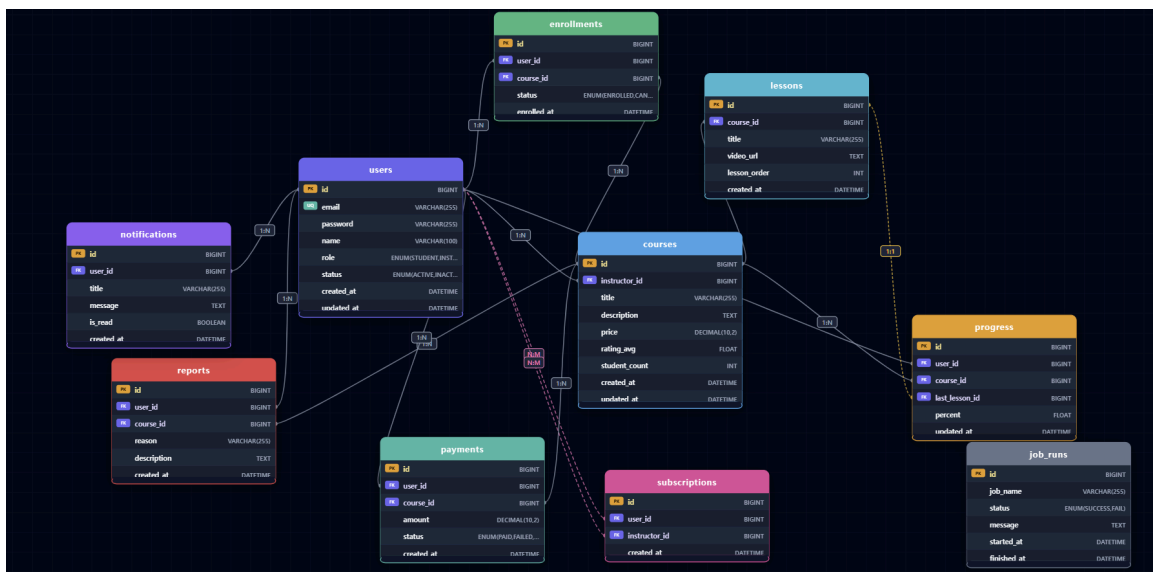
5. 강의 삭제 시나리오

1. 관리자가 강의를 삭제한다
 2. 강의 데이터가 삭제된다
 3. 해당 강의를 수강 중이던 사용자 목록을 조회한다
 4. 사용자에게 강의 삭제 알림을 발송한다
-

6. 운영 시나리오 (P3)

1. 새 강의 등록 이벤트가 발생한다
2. 디스코드 알림이 발송된다
3. 알림 실패 시 재시도한다
4. 오류 발생 시 Sentry에 기록된다
5. 운영자가 로그를 확인하고 대응한다

▼ ERD & sql문



-- DATABASE

```
CREATE DATABASE devclass;  
USE devclass;
```

-- USERS (사용자)

```
CREATE TABLE users (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL,
```

```

    name VARCHAR(100) NOT NULL,
    role ENUM('STUDENT', 'INSTRUCTOR', 'ADMIN') DEFAULT 'STUDENT',
    status ENUM('ACTIVE', 'INACTIVE') DEFAULT 'ACTIVE',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

```

-- COURSES (강의)

```

CREATE TABLE courses (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    instructor_id BIGINT NOT NULL,
    title VARCHAR(255) NOT NULL,
    description TEXT,
    price INT DEFAULT 0,
    rating_avg DECIMAL(2,1) DEFAULT 0,
    student_count INT DEFAULT 0,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    FOREIGN KEY (instructor_id) REFERENCES users(id)
);

```

-- LESSONS (강의 레슨)

```

CREATE TABLE lessons (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    course_id BIGINT NOT NULL,

```

```

    title VARCHAR(255) NOT NULL,
    video_url VARCHAR(500),
    lesson_order INT NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (course_id) REFERENCES courses(id) ON DE
LETE CASCADE
);

```

-- ENROLLMENTS (수강)

```

CREATE TABLE enrollments (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    course_id BIGINT NOT NULL,
    status ENUM('ENROLLED', 'CANCELLED') DEFAULT 'ENROLLE
D',
    enrolled_at DATETIME DEFAULT CURRENT_TIMESTAMP,

    UNIQUE KEY unique_enrollment (user_id, course_id),

    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (course_id) REFERENCES courses(id)
);

```

-- PROGRESS (강의 진도)

```

CREATE TABLE progress (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    course_id BIGINT NOT NULL,
    last_lesson_id BIGINT,

```

```

    percent INT DEFAULT 0,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPD
ATE CURRENT_TIMESTAMP,

    UNIQUE KEY unique_progress (user_id, course_id),

    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (course_id) REFERENCES courses(id),
    FOREIGN KEY (last_lesson_id) REFERENCES lessons(id)
);

```

-- REPORTS (강의 신고)

```

CREATE TABLE reports (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    course_id BIGINT NOT NULL,
    reason VARCHAR(100) NOT NULL,
    description TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

    UNIQUE KEY unique_report (user_id, course_id),

    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (course_id) REFERENCES courses(id)
);

```

-- NOTIFICATIONS (알림)

```

CREATE TABLE notifications (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,

```

```

        title VARCHAR(255),
        message TEXT,
        is_read BOOLEAN DEFAULT FALSE,
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

        FOREIGN KEY (user_id) REFERENCES users(id)
    );

-- SUBSCRIPTIONS (강사 알림 구독)

CREATE TABLE subscriptions (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    instructor_id BIGINT NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

    UNIQUE KEY unique_subscription (user_id, instructor_id),

    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (instructor_id) REFERENCES users(id)
);

-- PAYMENTS (결제)

CREATE TABLE payments (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    course_id BIGINT NOT NULL,
    amount INT NOT NULL,
    status ENUM('PAID', 'FAILED', 'REFUNDED') DEFAULT 'PAID',

```

```

        created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

        FOREIGN KEY (user_id) REFERENCES users(id),
        FOREIGN KEY (course_id) REFERENCES courses(id)
    );

-- JOB RUNS (스케줄러 로그)

CREATE TABLE job_runs (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    job_name VARCHAR(100) NOT NULL,
    status ENUM('SUCCESS', 'FAIL') NOT NULL,
    message TEXT,
    started_at DATETIME,
    finished_at DATETIME
);

-- INDEX (성능 최적화)

CREATE INDEX idx_courses_created_at ON courses(created_at);
CREATE INDEX idx_courses_instructor ON courses(instructor_id);
CREATE INDEX idx_enrollments_user ON enrollments(user_id);
CREATE INDEX idx_progress_user_course ON progress(user_id, course_id);
CREATE INDEX idx_reports_course ON reports(course_id);

```